

# Passive Pre-Association Wi-Fi Access-Point Selection

U.K.Arvindan  
EP21B006

**Abstract**— A Wi-Fi client in overlapping coverage must select an access point (AP) before it can directly measure the service that each AP would provide. Signal strength describes link quality, but not contention or load, and may therefore favor a nearby but congested AP. This project investigates whether a client can make a better decision using only passive observations collected before association without assistance from the network.

We define a controlled ns-3 simulator of multi-AP deployments to investigate this. The simulation of each topology of devices is replayed  $n$  times to a candidate STA to obtain the throughputs offered by each AP.

## I. PROBLEM FORMULATION

Throughput depends on channel contention, competing stations, transmission rates, retries, and the airtime consumed by neighboring basic service sets. Received signal strength reflects such effects poorly if at all. Those effects do, however, leave traces a client can hear without transmitting: the frames themselves, and the time the medium spends busy. This project asks whether such traces are enough: *can a client choose the AP that will provide the greatest throughput using only what its radio can passively observe before association?*

Existing load-aware methods [1], [2] make their load estimate after association, from AP-provided reports, or from a network controller. A client making its first choice has none of these: only beacons and some data frames, in which AP distance and offered traffic appear only through their effects.

Let  $\mathcal{A} = \{a_1, \dots, a_{N_{AP}}\}$  be the APs in one deployment and let  $O$  be the candidate station's passive observation before it joins any AP. If  $y_i$  is the throughput obtained after joining  $a_i$ , the desired decision is

$$a^* = \arg \max_{a_i \in \mathcal{A}} y_i, \quad (1)$$

using a decision function whose inputs are derived only from  $O$ .

We tackle this problem by predicting the value of  $y_i$  for each  $a_i$ .

It is worth noting that models that simply output a ranking of the available choices are viable as well. In the simulated data we ensure that we have true counterfactuals by deliberately verifying  $O$  to be identical across all  $N_{AP}$  runs of a given scenario. In a live network however, only one  $y_i$  is ever measurable: the station joins a single AP and observes that AP's throughput, while the values of  $y_j$  for every AP it did not join are not easily recoverable. For broader applicability,

and adaptation to reality, choice models are not investigated extensively here.

## II. SIMULATOR DESIGN AND ASSUMPTIONS

### A. Simulation Environment

The simulator is built on ns-3.45 and its Wi-Fi models [8]. The deployment is static and single-floor with no walls, the environment TGax assumes for its indoor small-BSS scenario [5]. All stations are immobile and remain at their seeded positions. Radios are homogeneous IEEE 802.11n devices at 5 GHz on 20 MHz channels with Minstrel-HT rate control [8].

Every radio is an ns-3 `SpectrumWifiPhy` attached to one shared `MultiModelSpectrumChannel`, on which a log-distance loss (exponent 3.0, with 46.6777 dB of reference loss at 1 m) is followed by Nakagami fading.

`SpectrumWifiPhy` is chosen over the simpler `YansWifiPhy` as the Yans model simulates exactly zero cross channel interference by treating each channel as an isolated layer. If `YansWifiPhy` is used, the channel occupancy given by CCA measurements becomes a powerful signal for models in a way that does not reflect reality.

### B. Count, Topology and Channel occupancy of APs

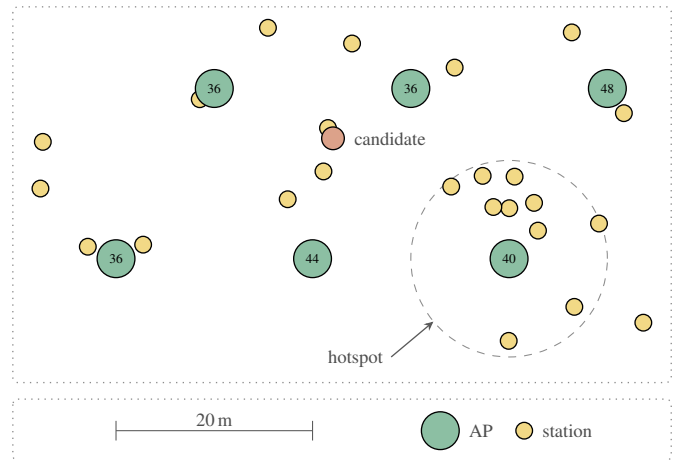


Fig. 1. An example simulated deployment.

Each scenario first draws 2, 3, 4, 6, or 8 APs. They sit on a triangular lattice 20 m apart, following [5], where it is reasoned that such a topology keeps the nearest-neighbor distance from varying with the number of APs. APs are confined to two rows of the lattice with the bottom row having  $\min(2, \lceil \frac{N_{AP}}{2} \rceil)$  APs.

Each AP then draws a channel uniformly at random from channels 36, 40, 44, and 48. Random assignment biases the data toward more in-channel contention and interference which could potentially make the problem harder than if we plan out the channels to be minimally interfering. However, TGax sets precedent for random assignment across four channels in its enterprise scenario, and across the non-overlapping set in its residential one [5], so this is the strategy adopted.

### C. Count, Topology of background STAs

The total number of stations is decided by independently drawing a count of 3, 4, or 5  $N_{AP}$  times and summing them.

A quarter of scenarios spread the stations uniformly. The rest declare one or more APs as "hotspots" around which stations are clustered. Such mixing of uniform and deliberately clustered regimes follows common practice in WLAN studies [3], [4], and is of particular import here because under uniform load every AP is about equally busy and the problem we examine collapses to signal strength in a fashion uncharacteristic of reality.

We draw the number of hotspots at random with fewer hotspots being more likely and the count capped at  $\lceil N_{AP}/3 \rceil$ . The area around a hotspot AP, a 10 m disc around it, is assigned a probability density 4 times that of non-hotspot areas. Each station's position is drawn from this distribution.

### D. Association and Load of background STAs

Each station associates with the AP whose received power is highest. This power is calculated with a log-distance term, then perturbed by a shadowing term: a random sample from a Gaussian of 5 dB standard deviation. Without the perturbation, stations form exact Voronoi partitions, which no real deployment does; 5 dB is the indoor shadowing deviation TGax specifies [5].

Every station then draws its own offered load, independently, from a log-normal distribution with median 2.25 Mbit/s and shape 0.9, capped at 25 Mbit/s, for a mean of 3.37 Mbit/s. Traffic volumes are log-normal rather than Gaussian [7], and the right skew is what puts most stations on a light load and a few on a heavy one without needing a mixture. The quartiles span the mix an AP actually carries, from voice and browsing up through high-definition video.

### E. Placement of candidate STA

The candidate station is placed uniformly at random over the same area as the background stations: the AP footprint plus a 10 m border.

### F. Data Capture

Background traffic starts at  $t = 0.5$  s. Each station's arrivals are Poisson at its own data rate, and its first packet is delayed by a phase drawn uniformly over one mean interval, without which, stations that share an AP and a rate transmit in lockstep for the whole run.

Passive recording runs from  $t = 2$  s to  $t = 8$  s, covering 6 s of loaded traffic. It starts 1.5 s after the traffic because Minstrel-HT needs about that long to converge, and until it does every scenario carries the same opening burst of retransmissions.

The candidate starts joining at  $t = 8$  s, the moment recording ends. After joining, the candidate offers 100 Mbit/s in 1250-byte packets until the run ends at  $t = 20$  s. That rate keeps it backlogged, so its measured throughput reflects the service the network gives it rather than the limit of its own traffic generator.

### G. Matched Counterfactual Replays

For each sampled topological arrangement of the candidate + environment, the experiment runs once per target AP. AP and station positions, background traffic, the candidate's position, and every pre-association device stay fixed; only the AP selected at  $t = 8$  s changes. A `topologySeed` fixes the deployment: station placement, channel selection, and the load offered by each station. Independently, `candidateSeed` fixes the candidate STA's position, and `rngSeed` controls all other randomness such as fading, contention, rate adaptation, and arrival times of background packets.

| Quantity           | Distribution                                         |
|--------------------|------------------------------------------------------|
| AP count           | uniform on {2, 3, 4, 6, 8}                           |
| AP channel         | uniform on {36, 40, 44, 48}, iid per AP              |
| Station count      | sum over APs of uniform on {3, 4, 5}                 |
| Spatial regime     | uniform with prob. 1/4, else hotspots                |
| Hotspot count      | $1 \dots \lceil N_{AP}/3 \rceil$ , fewer more likely |
| Hotspot field      | $4 \times$ station density inside a 10 m disc        |
| Station load       | log-normal, median 2.25 Mbit/s, shape 0.9, cap 25    |
| Association        | strongest received power + $\mathcal{N}(0, 5^2)$ dB  |
| Candidate position | uniform over the station area                        |

## III. DATASET CONSTRUCTION

### A. Recording and Projection

A deployed client scans across all channels that host APs in a rotating fashion. The simulation records more than any deployed client can be expected to observe by recording activity on every occupied channel simultaneously. We then project this complete record down to what a single scan would see.

Specifically, the candidate STA carries one receive-only scanner PHY per occupied channel, plus a separate PHY reserved for association. The scanner PHYs never transmit and never associate; the association PHY is parked on unused channel 149 until the decision time. At  $t = 8$  s the scanner PHYs move to the parking channel, and the association PHY moves into the channel of the AP it intends to associate with.

Two traces are recorded. The monitor-sniffer trace holds every frame a scanner decodes, with the start and end of the transmission carrying it, frequency, BSSID, transmitter, frame category, beacon and retry flags, length, RSSI, airtime, and PHY rate. Frames aggregated into one transmission share their start and end timestamps.

The PHY-state trace gives the fraction of each 1 ms interval in which carrier sense marked the channel busy. This clear-channel-assessment (CCA) measurement registers energy a

frame-only view misses: interference, and transmissions too distant to decode. Both thresholds are left at the ns-3 defaults:  $-62$  dBm of received energy from any source, or  $-82$  dBm from a signal whose preamble the PHY recognizes. Most driver stacks expose an equivalent channel-busy counter, though availability varies by platform.

The projection onto one passive radio is done as follows: One radio is assumed to rotate through the occupied channels in a certain randomly decided order, dwelling 110 ms on each with 0.15 ms to retune, and repeats the cycle for as much of the observation window as possible. A frame is kept only if its whole transmission falls inside a dwell after retuning: CCA intervals outside a dwell are discarded. A 110 ms dwell spans just over one 102.4 ms beacon interval, so each visit yields about one beacon from every AP on that channel.

RSSI is then perturbed by uniform error within  $\pm 3$  dB and quantized to 1 dB. Both steps run after ns-3 from a recorded scan seed, so scan policy and measurement noise can be varied without repeating a simulation.

### B. Training Data

The corpus used in the making of this report simulated 1,000 deployments, placing the candidate at five positions in each of them. Seven of the resulting 5,000 "scans" heard beacons from fewer than two APs and were dropped, having no choice to offer.

TABLE I  
THE CORPUS, AFTER PROJECTION ONTO ONE RADIO.

|                               |                  |
|-------------------------------|------------------|
| Deployments                   | 1,000            |
| Scans, of 5,000 drawn         | 4,993            |
| Simulated runs                | 22,925           |
| Discovered APs per scan       | 2 to 8           |
| Scans hearing every AP placed | 77.4%            |
| Throughput label              | 0 to 59.0 Mbit/s |
| median, mean                  | 5.3, 11.3 Mbit/s |
| exactly zero                  | 14.0% of rows    |

The label is right-skewed, its mean at twice its median, and the zero rows are due to APs that were heard but could not usefully be joined by a far away candidate.

Training splits of this dataset were drawn by topology rather than by scan. The five scans of a deployment differ only in where the candidate stands: the APs, their channels, the background stations and the traffic they offer are identical across them, so testing on a scan whose deployment also appears in training would score the model on something close to what it was fitted on. Deployments are dealt 60% to training, 20% to validation and 20% to test, each AP count dealt separately so that no split ends up with a disproportionate share of the largest or smallest deployments.

## IV. MODEL ARCHITECTURE

The model adapts a pretrained language model to this problem following the structure of NetLLM [9]: A learned encoder turns the observation into a sequence of vectors, a

frozen decoder-only backbone reads that sequence as input embeddings, and only low-rank adapters [10] on its attention projections are trained.

Every model in this section scores the *discovered APs* of a scan: those whose beacons it decoded at least once. An AP never heard leaves nothing in the observation to read from, so the arg max of Section I is in practice taken over the discovered APs rather than over every AP in the deployment. This is a limit of passive observation rather than of the models, and it costs little, since an AP too distant for its beacons to arrive is unlikely to be the one that would have served best.

### A. Embedding

The observation window contains several thousand frames and carrier sense datapoints, making attending over the scan frame by frame impractical. Further, several meaningful signals lay as aggregates or counts which would not be best captured by such a model.

Thus, we undertake the following process to vectorize our observation: First, the scan is cut into bins over time. Each bin is chosen to be of 110 ms, such that it coincides with channel dwells, keeping same-channel frames together. For each bin, we emit several *cells*. One cell is emitted per discovered AP: each of these only contains frames that carry its BSSID. We also emit one channel cell with all the frames of the dwell.

The model then learns a representation for each cell by allowing frames within the cell to attend amongst themselves. Frames carry 10 numerical values and a categorical flag; 9 numbers and the category are embedded into a 48-dimensional space with a 2 layer MLP. Non-linearity is needed here to capture relations between the frame characteristics, some of which are boolean, for attention to better relate frames to each other. The tenth value, time, is positionally encoded. The time from the start of the dwell is encoded as 8 sine-cosine pairs, with periods falling geometrically from 220 ms to 0.2 ms [11]. The fast end is about the airtime of one frame, below which there is nothing to distinguish; the slow end is twice the longest gap a cell can hold, so that the sine-cosine pair turns less than half a circle over any gap within a dwell and orders it unambiguously. The 8 pairs are appended to the aforementioned embedding, creating an overall embedding dimension of 64 for each frame.

Keeping time in its own columns separates the inner product of two frame tokens into a content term and a time term, and for sine-cosine pairs the time term depends only on the gap between the two start times. Xu *et al.* [14] take that separation as the requirement on a time encoding, asking its inner product to be a translation-invariant kernel, and derive from Bochner's theorem the functional forms that satisfy it, of which the sine-cosine pairs above are an instance. We adopt their construction.

The attended frames are then reduced to a single vector. We take two reductions and concatenate them: the mean over the cell's frames, and the output of one learned query attending over them, the pooling of the Set Transformer [16]. The mean reports the cell as a whole, while the query is free to weight a few frames above the rest. A cell hands over at most 512

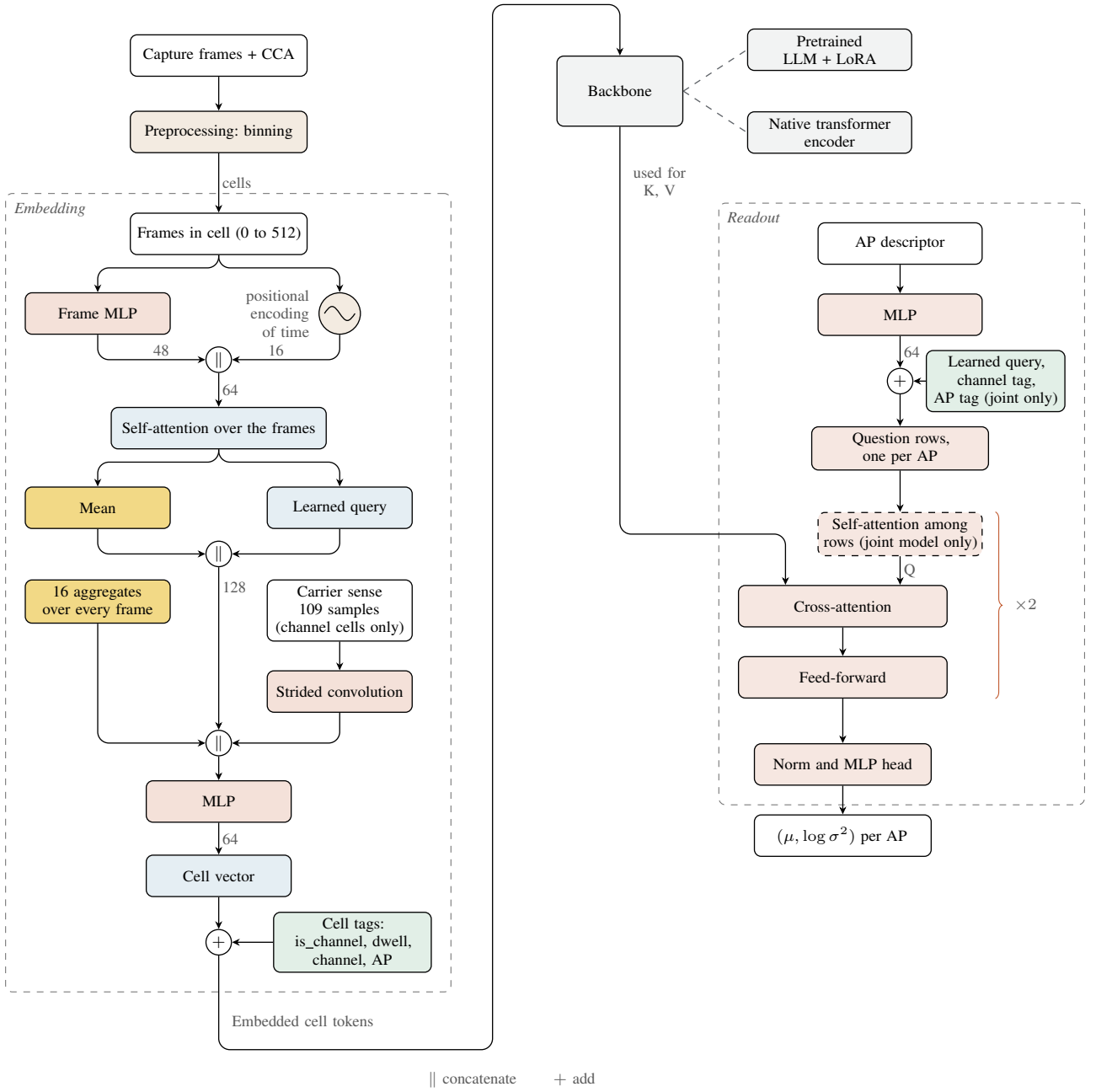


Fig. 2. Model architecture.

frames — a random subset while training, an even stride at evaluation — which bounds an attention cost that grows with the square of the frame count; the cap binds on 0.5% of cells.

Two exact summaries join the pooled vector. The first is 16 aggregates taken over every frame the cell held, including those the cap dropped. Table III lists the quantities that are aggregated here. Such counts and totals cannot be reported by the pooled set of vectors and are thus directly given to the model. The second summary belongs to channel cells alone: the 109 carrier-sense samples of the dwell, one per millisecond, read by a strided one-dimensional convolution. (One CCA sample is lost to radio retune)

A final MLP maps the concatenation of the summaries and the pooled vector to a single 64 dimensional vector leaving the encoder. (Figure 2). Both an AP and a channel cell share the frame embedding and the self-attention, but each keeps its own pooling query and its own output MLP, since only channel cells carry CCA signals, and an aggregate such as beacon level does not mean the same thing on the two and should not afforded the same mapping. A cell that heard nothing has nothing to pool and takes a learned vector in place of both reductions.

The resulting cell vector, although containing a summary of all its frames, mostly lacks identifiers to distinguish itself or relate to other cells. We add tags for the channel it was heard

on, whether it is an AP or channel cell, and its dwell in the sweep. Each tag is a 64 dimensional vector added onto the cell vector, its direction drawn once from a random orthogonal set and never trained. Tags from different sets are orthogonal, as are the tags within the cell type and channel sets. The dwell tags are instead sinusoidal in the dwell index, so that dwells close in time take similar tags. Each codebook has a trainable length shared by its entries, initially 0.5. This leaves the AP's identity, for which we take two approaches:

- Declare one AP as the target AP whose throughput we are interested in. We now code the identity relative to this AP: `TARGET_AP` for the target's own cells, `CO_CHANNEL_AP` for an AP sharing its channel, and `OTHER_AP` for the rest. These three roles use learned vectors. Inference runs once per target AP, with a different set of roles added to the cell vectors each time.
- Assign each AP in the scan a numerical identity and look up its fixed tag direction. The direction is shared by that AP's cells and its readout row; its length is learned. Identities are randomly relabelled for each training sample, so the model cannot attach a persistent meaning to an AP number. The table fixes the maximum number of APs handled at once, but permits all their throughputs to be predicted from one grid encoding.

### B. Backbone

The tagged cell vectors attend across the scan so that a cell describing one AP can incorporate the activity of other APs and channels. We consider two backbones for this step.

The first is a transformer encoder trained from scratch, with two blocks of four-head self-attention at width 64. Each block normalises its input before attention, adds the attended values to the input, then applies a feed-forward MLP with another normalisation and residual connection; that MLP widens to 256 and back. Each block also adds to its attention logits a learned scalar per head for the channel distance between the two cells, in 20 MHz steps. Every cell can attend to every observed cell; dwell order is supplied by the tags.

The second follows the adaptation of a pretrained language model described above. Linear projections map the cell vectors from 64 to 2048 dimensions, then back after Llama-3.2-1B has processed them. The backbone weights remain frozen, with LoRA adapters of rank 8 and  $\alpha = 32$  on the query and value projections, at dropout 0.05. Its attention mask is block causal, permitting cells to attend within their own dwell and to earlier dwells. Cells in the same dwell receive the same rotary position, since their order within that dwell carries no meaning [13].

The target-AP model runs the backbone separately for each AP, because changing the target changes the role tags on its input. The joint-AP model runs it once for the scan. AP identity tags in this model are assigned randomly within each sample and shared between an AP's cells and its readout; they provide a way to match observations without identifying an AP across deployments.

### C. Decoder

The backbone returns one vector per cell, while we need one prediction per AP. To obtain it, we construct a query row from the AP's descriptor, the whole-window summary of its beacons and its BSS in Table III. A two-layer MLP projects them to 64 dimensions, to which we add a learned query vector and the AP's channel tag. In the joint model, the row also receives the identity tag carried by that AP's cells.

These rows pass through two decoder blocks. Each block lets the rows attend to the backbone's cell vectors, using the rows as queries and the cells as keys and values, then applies a feed-forward MLP. Both operations have normalisation and residual connections. In the joint model, self-attention between AP rows precedes cross-attention, allowing each prediction to use what the other rows have learned. The target model has one row per backbone pass and omits this step.

A final MLP maps each row to a mean and log variance of standardised  $\log(1 + y_i)$ , where  $y_i$  is throughput in Mbit/s. Training begins with squared error on the mean and then uses Gaussian negative log likelihood to fit both outputs. The predicted means order the APs for selection.

## V. ARCHITECTURE CHOICES

### A. Frame Time and Cell Identity

The embedding above concatenates time with frame content, but adds identity to cell content. Both choices use a total width of 64. Concatenation reserves coordinates for each component; addition lets both use the full width, at the cost of superposing them [11].

At the cell level, addition lets the cell's frame and aggregate summaries occupy all 64 dimensions. Reserving 16 dimensions for tags would compress those summaries into 48 dimensions. The fixed cell-type and channel directions are distinct at input, while their learned lengths let the model set their weight against the raw cell content.

The separation also lasts only as long as subsequent layers preserve it. The value and output projections can write attended content into any coordinate, as can the feed-forward layers. After the first residual update, the reserved coordinates may contain both identity and content.

The sinusoidal argument at the frame level needs a similar qualification. For a time encoding  $p(\tau)$  with sine-cosine pairs at frequencies  $\omega_k$ ,

$$p(\tau_i)^\top p(\tau_j) = \sum_k \cos(\omega_k(\tau_i - \tau_j)). \quad (2)$$

This dot product depends only on the time difference. Attention, however, compares projected queries and keys. Writing their maps from the positional coordinates as  $U_Q$  and  $U_K$ , the positional term is  $p(\tau_i)^\top U_Q^\top U_K p(\tau_j)$ , which need not retain that property. Concatenation alone neither constrains these projections nor removes the content–position cross terms. TUPE [15] treats content and position with separate attention terms; reserving input coordinates does not implement that separation.

Addition offers a different allocation of the same width: all 64 coordinates can represent frame content, and time can use 32 sine-cosine pairs rather than 8. More pairs sample the chosen frequency range more densely, though that need not improve the task. Whether that extra capacity improves AP selection remains to be established, so the present model retains concatenation at the frame level.

### B. Identity in the Attention Logits

Two cells' channel tags already say whether they share a channel. We can also supply that relationship directly to attention. In the custom encoder, each head can learn a scalar for each channel separation, measured in 20 MHz steps, and add it to the scaled query-key score before softmax. The decoder can use its own channel-separation table and an additional scalar indicating whether the cell belongs to the AP being scored. These are learned preferences rather than masks: every observed cell remains available to attention.

T5 [17] and Swin [18] use learned relative-position biases in attention, while Graphormer [19] supplies graph relationships through such biases. Here, however, the relationships are also present in the tags. This raises the question of whether the explicit biases provide a useful constraint or duplicate something attention can learn.

To see the overlap, write an input as content plus tag,  $x_i = c_i + t_i$ . Let  $W_Q$  and  $W_K$  be one head's query and key maps. Temporarily omitting normalisation, projection biases and the attention scale, set  $M = W_Q^T W_K$ . Then

$$\begin{aligned} q_i^T k_j &= c_i^T M c_j + c_i^T M t_j \\ &\quad + t_i^T M c_j + t_i^T M t_j. \end{aligned} \quad (3)$$

The last term assigns a score to a pair of tags independently of content. It can therefore express preferences such as matching identities, which are also supplied by an explicit bias. In cross-attention the same expansion applies with row content and tags on the query side and cell content and tags on the key side.

This overlap does not establish equivalence. A tag-derived table is constrained by the fixed tag directions and the head's projections; an explicit table directly ties all pairs with the same relationship to one parameter. Our blocks also normalise the combined token before projecting it, and later layers receive attended values. We therefore keep the learned channel-distance bias in the encoder and both the channel-distance and own-AP biases in the decoder.

## VI. RESULTS

Table II shows the available results on the expanded 1,000-topology corpus. It contains 4,993 usable scans, of which 1,013 belong to fold 0. The reference rules, tabular predictors, target-AP model and joint-AP model were evaluated on all five topology folds, with one fold for test, the next for validation and the remaining three for training. Their entries are the mean and sample standard deviation of the five fold scores. The remaining entries were evaluated on fold 0 alone, so each is an individual score rather than a cross-fold estimate. Fold 0 of the target-AP and joint-AP models appears among them,

so that the single-fold entries can be read against the same held-out topologies.

MAE is absolute throughput error. Top-1 is the fraction of scans on which the selected AP is best; regret is the throughput lost by that choice; Spearman correlation measures the ordering of discovered APs within each scan. The rules do not predict throughput, so they have no MAE, and neither do the models fitted to the ranking loss, whose output orders a scan's APs rather than estimating what each would deliver. Ridge regression, random forests and gradient boosting use 32 window features and choose their hyperparameters on validation.

The joint-AP model has the lowest observed five-fold mean regret, 0.422 Mbit/s, followed by target AP at 0.444 and gradient boosting at 0.448. The fold-to-fold variation does not establish that these differences generalise. Both cell transformers use fixed shared tags and raw cell content; target AP keeps learned relative-role embeddings, while joint AP uses fixed AP-identity directions. The comparison with a LoRA-adapted language model remains open on this corpus.

## VII. FURTHER STUDY

### A. AP-Advertised Load

The present model estimates load from transmissions and carrier sense at the candidate station. An AP may also advertise BSS Load information in its beacons. Adding this information would retain passive pre-association operation, while giving the model a view from the AP that the client's observations need not reproduce. We should compare predictions with and without these fields, including cases where they are absent or stale. This would also permit a closer comparison with selection methods that use AP-reported load.

### B. Traffic That Changes During the Scan

Each background station currently keeps a fixed mean offered rate throughout a run. Poisson arrivals and wireless contention introduce variation, but the simulator does not model sustained changes in demand. Repeated observations therefore mostly refine an estimate of the same traffic regime.

Bursty sessions, changes in offered rate, and stations starting or stopping traffic would make the order and recency of observations more consequential. A useful extension would vary both the size of these changes and their time scale relative to scanning and association. The throughput measurement horizon should then be varied too: a long average after association may hide a short period of poor service that matters to the client. Matched replays must preserve the same background demand schedule across the alternative AP associations.

### C. Shorter Observation Windows

A client may not be willing to spend six seconds observing a network, and under changing traffic the earliest observations may already be stale. We should compare shorter windows, including one second, ending at the same association time. Descriptors, aggregates and the set of discovered APs must be rebuilt from each window so that no information from discarded observations reaches the model.

TABLE II  
1,000-TOPOLOGY CORPUS RESULTS. MAE AND MEAN REGRET ARE IN MBIT/S.

| Model                                 | MAE ↓             | Top-1 ↑           | Mean regret ↓      | Spearman ↑        |
|---------------------------------------|-------------------|-------------------|--------------------|-------------------|
| <i>Reference rules: five folds</i>    |                   |                   |                    |                   |
| Random choice                         | —                 | $0.287 \pm 0.001$ | $14.426 \pm 0.590$ | —                 |
| Strongest RSSI                        | —                 | $0.621 \pm 0.025$ | $5.263 \pm 0.676$  | $0.615 \pm 0.022$ |
| Least busy channel                    | —                 | $0.525 \pm 0.033$ | $6.431 \pm 0.329$  | $0.391 \pm 0.042$ |
| RSSI minus busy                       | —                 | $0.803 \pm 0.020$ | $1.386 \pm 0.138$  | $0.754 \pm 0.024$ |
| <i>Tabular predictors: five folds</i> |                   |                   |                    |                   |
| Ridge regression                      | $4.892 \pm 0.360$ | $0.827 \pm 0.016$ | $1.031 \pm 0.070$  | $0.803 \pm 0.008$ |
| Random forest                         | $2.396 \pm 0.030$ | $0.878 \pm 0.014$ | $0.508 \pm 0.058$  | $0.865 \pm 0.007$ |
| Gradient boosting                     | $2.264 \pm 0.082$ | $0.883 \pm 0.015$ | $0.448 \pm 0.078$  | $0.866 \pm 0.007$ |
| <i>Cell transformers: five folds</i>  |                   |                   |                    |                   |
| Target AP                             | $2.452 \pm 0.414$ | $0.887 \pm 0.020$ | $0.444 \pm 0.093$  | $0.879 \pm 0.016$ |
| Joint AP                              | $2.032 \pm 0.067$ | $0.888 \pm 0.009$ | $0.422 \pm 0.040$  | $0.879 \pm 0.015$ |
| <i>Fold 0 only</i>                    |                   |                   |                    |                   |
| Binned sequence                       | 2.218             | 0.881             | 0.533              | 0.865             |
| Target AP                             | 2.221             | 0.891             | 0.465              | 0.888             |
| Joint AP                              | 2.140             | 0.890             | 0.435              | 0.871             |
| Target AP, LoRA backbone              | 2.038             | 0.886             | 0.462              | 0.885             |
| Joint AP, LoRA backbone               | 2.163             | 0.885             | 0.485              | 0.880             |
| Joint AP, ranking loss, $T = 1$       | —                 | 0.888             | 0.480              | 0.886             |
| Joint AP, ranking loss, $T = 0.25$    | —                 | 0.889             | 0.423              | 0.872             |
| Joint AP, ranking loss, $T = 0$       | —                 | 0.871             | 0.602              | 0.843             |

Five-fold entries show mean  $\pm$  standard deviation; fold-0 entries are single scores and need not represent cross-fold performance.

Under the current fixed-rate traffic, a short window may recover much of the same information with fewer samples. That remains a hypothesis: fewer beacon receptions and less evidence of contention can still reduce reliability. Under changing demand, a shorter window also trades statistical precision against recency. Selection regret should be accompanied by AP-discovery coverage, distinguishing a poor choice among heard APs from a better AP that the shorter scan never discovered.

#### D. Variation Beyond the Simulated Deployment

The present corpus fixes radio capabilities and channel width, places APs on a regular lattice, and has no walls or device motion. Irregular layouts, obstructions and mobility would test whether the model can distinguish a change in propagation from a change in load. Mixed radio capabilities and channel widths would test whether relationships learned under homogeneous 802.11n devices transfer to networks with different service rates. These variations should include held-out environments, rather than only more samples from the same parameter distributions.

Finally, real monitor-mode captures would test which simulated observations are available and reliable on client hardware. Carrier-sense counters, PHY metadata and frame reception may differ between devices. Such a study would also face a different labelling constraint: a real client observes service from the AP it joins, while the simultaneous alternatives remain unknown. Controlled repeated measurements can help, but should not be presented as the exact counterfactuals supplied by the simulator.

#### REFERENCES

- [1] P. Dely, A. Kassler, L. Chow, N. Bambos, N. Bayer, H. Einsiedler, and C. Peylo, “BEST-AP: Non-intrusive estimation of available bandwidth and its application for dynamic access point selection,” *Computer Communications*, vol. 39, pp. 78–91, 2014, doi: 10.1016/j.comcom.2013.11.006.
- [2] T. Adame, M. Carrascosa, B. Bellalta, I. Pretel, and I. Etxebarria, “Channel load aware AP/Extender selection in home WiFi networks using IEEE 802.11k/v,” *IEEE Access*, vol. 9, pp. 30095–30112, 2021, doi: 10.1109/ACCESS.2021.3059473.
- [3] W. Wong and S.-H. G. Chan, “Distributed joint AP grouping and user association for MU-MIMO networks,” in *Proc. IEEE INFOCOM*, 2018, pp. 252–260, doi: 10.1109/INFOCOM.2018.8486284.
- [4] Y. Zhang and L. Dai, “Joint optimization of placement and coverage of access points for IEEE 802.11 networks,” in *Proc. IEEE ICC*, 2020, pp. 1–7, doi: 10.1109/ICC40277.2020.9149098.
- [5] IEEE 802.11 Task Group ax, “TGax simulation scenarios,” IEEE 802.11-14/0980r16, 2016. [Online]. Available: <https://mentor.ieee.org/802.11/dcn/14/11-14-0980-16-00ax-simulation-scenarios.docx>
- [6] 3GPP, “Evolved Universal Terrestrial Radio Access (E-UTRA); Further advancements for E-UTRA physical layer aspects,” TR 36.814 V9.0.0, Mar. 2010.
- [7] M. Alasmar, R. Clegg, N. Zakhleniuk, and G. Parisi, “Internet traffic volumes are not Gaussian — they are log-normal: An 18-year longitudinal study with implications for modelling and prediction,” arXiv:2007.10150, 2020.
- [8] ns-3 Project, *ns-3 Model Library, Release 3.45*, 2025. [Online]. Available: <https://www.nsnam.org/docs/release/3.45/models/>
- [9] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, “NetLLM: Adapting large language models for networking,” in *Proc. ACM SIGCOMM*, 2024, pp. 661–678, doi: 10.1145/3651890.3672268.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. ICLR*, 2022.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [12] A. Grattafiori et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.
- [13] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, art. 127063, 2024, doi: 10.1016/j.neucom.2023.127063.
- [14] D. Xu, C. Ruan, S. Kumar, E. Korpeoglu, and K. Achan, “Self-attention with functional time representation learning,” in *Proc. NeurIPS*, 2019, pp. 15915–15925.
- [15] G. Ke, D. He, and T.-Y. Liu, “Rethinking positional encoding in language pre-training,” in *Proc. ICLR*, 2021.

- [16] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, “Set Transformer: A framework for attention-based permutation-invariant neural networks,” in *Proc. ICML*, 2019, pp. 3744–3753.
- [17] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [18] Z. Liu *et al.*, “Swin Transformer: Hierarchical vision transformer using shifted windows,” in *Proc. IEEE/CVF ICCV*, 2021, pp. 10012–10022.
- [19] C. Ying *et al.*, “Do transformers really perform bad for graph representation?” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 28877–28888, 2021.
- [20] M. Tan, M. A. Merrill, V. Gupta, T. Althoff, and T. Hartvigsen, “Are language models actually useful for time series forecasting?” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2406.16964>

## APPENDIX A MODEL INPUTS

Every number the models read, in the three groups they arrive in. Rates and fractions are normalized by the time actually spent listening on the relevant channel, not by the observation window. A  $\log_{10}$  suffix marks a count or duration stored as  $\log(1 + x)$ .

TABLE III  
THE MODEL INPUTS. FRAME FEATURES ARE READ ONCE PER DECODED FRAME, AGGREGATES ONCE PER CELL OVER EVERY FRAME IT HELD, AND DESCRIPTORS ONCE PER DISCOVERED AP OVER THE WHOLE WINDOW.

| Column                                  | Meaning                                                                                                    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Frame</b>                            |                                                                                                            |
| offset_in_dwell                         | Where in the 110ms dwell the transmission began, 0 to 1; encoded as sine-cosine pairs rather than embedded |
| gap_loglp                               | Idle time on this channel before it                                                                        |
| rss_i_dbm                               | Received level                                                                                             |
| airtime_loglp                           | The frame's own airtime, not that of the transmission carrying it                                          |
| length_loglp                            | Length in bytes                                                                                            |
| rate_loglp                              | PHY rate                                                                                                   |
| is_beacon, is_retry                     | Beacon and retry flags                                                                                     |
| from_ap                                 | The BSSID itself sent it, not one of its stations                                                          |
| of_valid_ap                             | Carries a discovered AP's BSSID                                                                            |
| category                                | Management, control or data; embedded rather than numeric                                                  |
| <b>Cell aggregate</b>                   |                                                                                                            |
| frames_loglp, bytes_loglp               | Frames and bytes the cell held                                                                             |
| airtime_fraction                        | Their own airtimes summed, over the time listened                                                          |
| busy_fraction                           | Time with a decoded transmission on air, overlaps counted once                                             |
| data_fraction, mgmt_fraction            | Share of frames by category                                                                                |
| retry_fraction                          | Share marked retry                                                                                         |
| transmitters_loglp                      | Distinct transmitters; on an AP cell its own is left out                                                   |
| beacons_loglp                           | The AP's beacons; 0 on a channel cell                                                                      |
| beacon_rssi_mean                        | Their level; missing on a channel cell                                                                     |
| rssi_mean, rssi_max                     | Over every frame, its stations' as well as the AP's                                                        |
| data_rate_loglp                         | Mean PHY rate of the data frames                                                                           |
| longest_gap_fraction                    | Longest stretch with nothing decoded, over the time listened                                               |
| cca_mean, cca_max                       | Carrier sense over the dwell; 0 on an AP cell                                                              |
| <b>AP descriptor</b>                    |                                                                                                            |
| beacon_rssi_{mean, std, min, max, last} | Beacon level over the whole window: mean, spread, extremes, and most recent                                |
| beacons_loglp                           | Beacons decoded                                                                                            |
| bss_airtime_fraction                    | Its BSS's summed airtime, over the time listened on its channel                                            |
| bss_transmitters_loglp                  | Distinct stations heard sending in its BSS                                                                 |
| cochannel_discovered_aps                | Other discovered APs on its channel                                                                        |
| n_discovered_aps                        | Discovered APs in the scan                                                                                 |